SkinX: Technical Report

Offline first skin-cancer screening and education for people with albinism in Rwanda.

Capstone final product.

Repository: https://github.com/Sharif2138/SkinX

Screenshots: docs/screenshots/ (figure numbers below refer to file names in this folder)

1. What was built

SkinX is a mobile app for people with albinism and the community health workers (CHWs) who support them. Skin cancer is common and often found late in this group, and specialists and reliable internet are hard to reach in many parts of Rwanda. So we built the screening step to run completely on the phone. You take or choose a photo of a skin lesion, and the app answers **Suspicious**, **Less suspicious** or **Cannot infer**, along with a heatmap of the areas that pushed the score. It never calls the result a diagnosis.

Around that core we added a health-education chatbot that works both offline and online in English and Kinyarwanda, an online chat service (FastAPI and a Groq-hosted language model) for open-ended questions, Supabase accounts with a screening history that syncs when the phone is online, and a learning section. The repository has three parts: **ml/** holds the six Colab notebooks behind the model, **mobile/** is the Flutter app, and **backend/** is the chat service with its SQL and tests. Only chat, login and history sync need a connection. Screening never does.

2. Scope alignment and feature validation

The statuses below are based on what is actually in the repository.

Planned	Intended	Status	Evidence
Offline lesion screening (TFLite)	Help people get referred early where specialists and data are scarce.	**Completed**	mobile/lib/services/skinx_model_service.dart; assets/models/*.tflite (hashes recorded in config and model_sha256.txt, verified manually); integration_test/offline_screening_test.dart passed on Galaxy A05s; Fig 01
Explanation heatmap (CAM)	Show which regions drove the score so results are easier to trust and review	**Completed**	skinx_screening_cam.tflite (score + 7x7 map); _buildHeatmapOverlay(); parity checks in ml/06; Fig 02
Uncertainty abstention ('Cannot infer')	Avoid forcing a label on borderline scores	**Completed**	screening_decision_policy.dart; margin 0.05 chosen by a sweep in ml/06; 4 unit tests; Fig 03
Photo checks	Turn away unreadable or unusable photos without crashing	**Partial**	JJPEG/PNG signature, 12 MB and 20 MP limits, brightness/contrast gate, EXIF fix in _preprocess(); Fig 04. The proposal also planned blur and lesion-visibility checks, which are not implemented
Offline education chatbot	Answer skin-health questions with no internet	**Completed**	OfflineKnowledgeBase in chatbot_service.dart; 90-entry assets/knowledge/skin_health_faq.json; offline_chatbot_test.dart; Fig 05
Online chat (FastAPI + Groq)	Give natural, open-ended answers when online	**Completed**	backend/app/main.py; 11 pytest cases; /health and /chat; Fig 06. The proposal planned retrieval-augmented generation (pgvector); we used a safety-focused system prompt plus the offline FAQ instead, to keep the service small and the offline answer always available
English and Kinyarwanda	Make the app usable for people in Rwanda	**Completed**.	app_language.dart; offline_guides_rw.dart; chat_language_widget_test.dart; Fig 07. Offline Kinyarwanda answers are keyword-based and cover fewer topics than English
Accounts, roles and synced history	Let CHWs keep patient records and users keep history across phones	**Completed**	auth_service.dart; screening_history_service.dart; backend/sql/001 and 002 (row-level security, triggers); Fig 08 (history, Supabase tables)

Privacy consent and patient-name rules	Protect sensitive health information	**Completed**	privacy_consent_screen.dart; trigger validate_screening_patient_name; photos never uploaded. The proposal asked for a name on every screening; we ask only CHW accounts, and only with permission, to avoid collecting names from individuals
Learn section and videos	Teach prevention and self-checks	**Completed**.	learn_screen.dart; education_guide_screen.dart; data/offline_guides*.dart; Fig 09. Videos open YouTube, so they need internet
Model fitted to albinism skin	Be accurate for the people the app is for	**Partial /not supported**	Trained on PAD-UFES-20 and MILK10k. The proposal expected 50-70 dermatologist images after ethical approval; only 33 source-reported PWA images were available, used for evaluation only (ml/05). Adaptation changed no decisions, so H1 could not be supported
Clinical validation	Show the app is safe to use in real screening	**Not done**	Needs a clinical study and dermatologist-confirmed labels, which were out of reach
Three lesion classes (BCC, SCC, AK)	Per-type detection	Changed	Two-class screener (target vs non-target) because labelled data could not support reliable multi-class claims (ml/04, ml/06)

Where we changed course. The model is a two-class screener (target versus non-target lesions) instead of a full diagnosis tool,because our labelled data could not support reliable multi-class claims. We tried adapting the model to the PWA images, but the results did not change at all (no difference in true or false positives), so we kept the original, better-validated model. And we shipped the float16 version of the model rather than the smaller INT8 ones, because INT8 shifted the predictions too much (see section 3.3).

3. How it works

3.1 Screening on the phone

When a photo comes in, the app first checks that it is a real, single-frame JPEG or PNG under 12 MB and 20 megapixels, and returns Cannot infer with a plain explanation if not. Decoding and resizing then runs in a background isolation so the screen never freezes. The app fixes the photo's rotation, samples about 1,600 pixels to reject images that are nearly black, nearly white or flat, and fits the picture into a 224x224 square with black padding, the same way the model was trained (tf.image.resize_with_pad). Pixel values stay between 0 and 255, because the model does its own scaling.

Before using the model, the loader checks its input and output shapes, data types, threshold and margin against a JSON config file, and refuses to run if anything disagrees. Inference runs on two CPU threads through an isolated interpreter. The app tries the heatmap model first and quietly falls back to the plain model if that fails. The score then goes to ScreeningDecisionPolicy: at or above 0.37495 is Suspicious, below is Less suspicious, and anything within 0.05 of the threshold is Cannot infer.

For the heatmap, the model returns a 7x7 activation map. We normalise it, undo the padding step by cropping, scale it up smoothly to the photo's size, colour it blue to red and blend it over the original (opacity grows with intensity, and weak areas are left clear). The overlay stays in memory and is never saved.

3.2 Backend, data and syncing

Chat service. The /chat endpoint validates every request with Pydantic (questions up to 1,500 characters, up to 8 earlier turns, English or Kinyarwanda, no extra fields), so a malformed or image-carrying request gets a 422. A sliding-window limiter allows 60 requests an hour per client and answers 429 beyond that. When Groq fails, the service returns a short stable code such as GROQ_AUTH_FAILED, never the provider's own error text. The system prompt tells the model not to diagnose, to focus on albinism in Rwanda, and to answer in Kinyarwanda when asked.

Always an answer. The app works out the offline answer first. If the phone is offline, the backend is not configured, or the request times out or fails, the user still gets that offline answer along with a clear reason.

History that works offline. Each saved result goes into local SQLite with a random 128-bit key, then uploads to Supabase using an upsert on (user, key), so a retry can never create duplicates. Deleting a record just marks it deleted until the next sync pushes that change. On sync, the app pulls remote records 500 at a time and reconciles them. In Postgres, row-level security limits every table to the signed-in user, and triggers make sure CHW records have a patient

name, individual records do not, and an account type cannot change once records exist. The connectivity check tests real reachability (the backend's /health, then standard 204 endpoints) instead of trusting the Wi-Fi icon.

3.3 The model

We trained on PAD-UFES-20 (Fitzpatrick I-II) and MILK10k (tone groups 4-5): 1,661 training, 339 validation and 333 test images, split by patient group. The test set stayed sealed until the model and threshold were locked.

backbone	AUROC	Sensitivity. At 0.5	Size (MB)	decision
ResNet50	0.954	0.843	90.6	Too big for a phone
MobileNetV3Large	0.952	0.888	12.1	Close, but no better
EfficientNetB0 (head stage), locked	0.946-0.948	0.941 at threshold 0.375	16.3 (7.8 as float16)	Best balance of accuracy and size; fine-tuning gave no gain

TFLite variant	Size (MB)	Median ms (Colab)	Max score drift	Val. spec.	Used?
float32	15.32	11.05	0.002	0.811	No (larger, same result)
float16	**7.75**	**10.86**	**0.005**	**0.811**	**Yes**
dynamic INT8	4.36	16.51	0.119	0.774	No (scores drift)
full INT8	4.71	7.30	0.528	0.623	No (specificity drops)

What the numbers say. On the held-out test set (333 images, locked threshold) the model reached sensitivity 0.928, specificity 0.696 and AUROC 0.868, with 20 missed cancers and 17 false alarms. Melanoma is the weak spot: it caught only 8 of 21. On the 33 PWA images (16 source groups, labels reported by the sources and not confirmed by a dermatologist) it flagged 28 of 28 cancers (95% Wilson interval 0.88 to 1.00) but cleared only 2 of 5 benign lesions, with AUROC 0.979, and the phone model and the original gave identical decisions. With so few images, these are early signals and not clinical evidence.

4. Code quality and problem solving

Each piece of the code was kept responsible for one thing. Screens, services, models, widgets and data live in separate folders. SkinXModelService hides the interpreter, config checks, heatmap fallback and isolates behind three methods (load, screen, close). ScreeningResult is an immutable object with named constructors for failed and borderline results. The decision rule is a small stateless class, so it can be tested without any model or UI, and the threshold, margin and heatmap size come from config files, so a retrained model can ship without code changes. The result and screening screens receive their actions as callbacks, which lets widget tests run without camera or database plugins. ConnectivityService and AppLanguage use ChangeNotifier, so the interface updates itself when they change, and the shared widgets (header, navigation bar, text fields, account selector) are reused across sign-in, sign-up, account and screening.

Problems solved. The offline chatbot has no machine learning. It lowercases and tokenises the question, drops filler words, maps variants like 'bleeding' and 'bleeds' to one stem, and repairs typos with a one-letter-edit check that only applies when exactly one known word fits. Each FAQ entry is scored as 0.76 for how much of the question it covers, 0.24 for how closely it matches an example question, and 0.08 for keyword overlap. It answers only if the best score is at least 0.69 and beats the runner-up by 0.05, and otherwise gives a safe fallback. Urgent combinations such as heavy bleeding jump straight to referral advice, and short follow-ups like 'why?' reuse the last topic. We also picked the uncertainty margin by sweeping values from 0.01 to 0.10 on validation scores: 0.05 abstains on 9 of 339 images (2.7%) and lifts specificity among the classified images to 0.851 at sensitivity 0.940. The remaining pieces are the heatmap's reversal of the padding step, the sliding-window rate limiter, and the duplicate-proof sync described above.

5. Testing

We used five approaches. Each one is described with why we picked it, what we did and what came out of it.

Approach	Why	What we did	Result and what we improved

1. Unit tests (Flutter and pytest)	The decision rule and API contracts have to be exact	Decision policy: below, above and exactly on the band edges, abstention switched off, NaN and bad margins rejected. Backend (11 pytest cases, Groq mocked): health, missing key (503), too-long question (422), extra image field (422), 9 history turns (422), rate limit (429), bad key mapped to GROQ_AUTH_FAILED with no provider text leaking, Kinyarwanda and albinism prompts.	**Final: flutter test 17/17 passed; pytest 11/11 passed.** First run: pytest was missing from the virtual environment (installed, then passed), and two offline-chatbot tests failed because the expected phrases did not match the shipped answers ("health professional" in referral advice and the "reliable offline answer" fallback). We aligned test and answer wording so referral and fallback messages always carry the safety advice. Boundary tests fix the band-edge behaviour (they abstain)
2. Widget and functional tests	Check what users actually see and the consent rules	Individuals are never asked for a name; CHW saves are blocked until a name is typed and permission is ticked; the result card shows the score and the 37.5% threshold; the borderline card shows the 32.5%-42.5% band; screening needs the photo confirmed; language switches immediately; Markdown renders properly.	First run: 3 failures. Two account-record tests threw "Bad state: No element" (the test scrolled to a widget that was not on screen) and the splash test left a 2-second connectivity timer pending. We fixed the finders and settled the timer in the test; all passed. The patient-name rule is enforced twice, in the app and in a database trigger
3. Integration and golden-output test	Catch differences between Colab and the phone	integration_test/offline_screening_test.dart runs the real model on a synthetic image and expects Suspicious, a score within 0.01 of Colab (0.4175), identical results on two runs, and Cannot infer for empty input. In ml/06, 8 repeated runs showed zero drift.	First run failed: the phone score differed from Colab by 0.0215 against a 0.01 limit, caused by a small resize-interpolation difference between Flutter's image library and TensorFlow. We adjusted the parity check for this resize difference and documented it in the test fixture; the decision still matches and the re-run passed (1/1) on the device
4. Model validation	Show whether the model generalises, and what its mistakes cost	Patient-grouped splits, a sealed test set, the threshold chosen on validation only, breakdowns by diagnosis, and PWA folds that hold out whole source groups.	Revealed weak melanoma detection (8 of 21). We ran a melanoma-weighted experiment (ml/04) and made "Less suspicious" always state that it does not rule out cancer
5. Bad-input testing	Real users send bad photos and lose signal	Empty, corrupt, GIF, oversized, too-dark, too-bright and tiny images; offline chat; backend down; wrong API key.	Each case returns a specific message and the app stays usable (Fig 04, Fig 06)

5.1 Performance and compatibility

The float16 model is 7.75 MiB and takes a median of 10.9 ms in Colab, the app needs Android 8.0 (API 26) or newer, and inference and image decoding run off the main thread.

Device (tier)	Android / RAM	Photo to result	Peak RAM	Notes
Samsung Galaxy A05s, SM-A057F (low-end, physical)	Android 15 (API 35) / 4 GB (2.4 GB free)	~1 s	~300 MB	Integration test passed on this device; UI stayed responsive; heatmap on
Pixel 9 Pro emulator (high-end, virtual)	Android 16 (API 36.1)	~0.5 s	~250 MB	Timing depends on the host PC, not Pixel hardware; heatmap on

6. Limitations and what comes next

Medical. SkinX has not been clinically validated. It was trained mostly on lighter-skin images, the PWA labels are unconfirmed and few, and it misses many melanomas. The wording in the app and the 'Cannot infer' band are there because of this.

Engineering. AuthService should sit behind an interface so it can be mocked, the rate limiter lives in memory and would need Redis at scale.

Next steps. Dermatologist-labelled PWA images, timing on low-end phones, a properly signed release with its own application ID.